# Experiment 2

**Aim:** Write a Python program to implement Breadth First Search (BFS) algorithm for a given graph.

**Code:**

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])

    visited.add(start)

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

bfs(graph, 'A')
```

---

# Experiment 3

**Aim:** Write a Python program to implement Depth First Search (DFS) algorithm for a given graph.

**Code:**

```
def dfs(graph, node, visited=None):
    if visited is None:
        visited = set()

    visited.add(node)
    print(node, end=" ")

    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)

graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

dfs(graph, 'A')
```

---

# Experiment 4

**Aim:** Write a Python program to implement Best First Search algorithm for a given graph.

**Code:**

```
import heapq

def best_first_search(graph, start, goal, heuristic):
```

```
    visited = set()
    priority_queue = []

    heapq.heappush(priority_queue, (heuristic[start], start))

    while priority_queue:
        h_value, current = heapq.heappop(priority_queue)

        if current in visited:
            continue

        visited.add(current)

        print(current, end=" ")

        if current == goal:
            print("\nGoal reached!")
            return

        for neighbor in graph[current]:
            if neighbor not in visited:
                heapq.heappush(priority_queue, (heuristic[neighbor], neighbor))

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

heuristic = {
    'A': 5,
    'B': 3,
    'C': 4,
    'D': 2,
    'E': 6,
    'F': 1
}

best_first_search(graph, 'A', 'F', heuristic)
```

---

# Experiment 5

**Aim:** Write a Python program to implement A* Search Algorithm.

### Code:

```
import heapq

def a_star_search(graph, start, goal, heuristic):
    priority_queue = []
    visited = set()

    heapq.heappush(priority_queue, (heuristic[start], 0, start, [start]))

    while priority_queue:
        f_value, g_value, current, path = heapq.heappop(priority_queue)

        if current in visited:
            continue

        visited.add(current)

        print(current, end=" ")

        if current == goal:
            print("\nGoal reached!")
            print("Path:", " -> ".join(path))
            return

        for neighbor, cost in graph[current]:
            if neighbor not in visited:
                g_n = g_value + cost
                f_n = g_n + heuristic[neighbor]
                heapq.heappush(priority_queue, (f_n, g_n, neighbor, path + [neighbor]))
```

```
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 1), ('E', 5)],
    'C': [('F', 2)],
    'D': [],
    'E': [],
    'F': []
}

heuristic = {
    'A': 6,
    'B': 4,
    'C': 4,
    'D': 0,
    'E': 2,
    'F': 1
}

a_star_search(graph, 'A', 'D', heuristic)
```

---

# Experiment 6

**Aim:** Write a Python program to implement Travelling Salesman Problem.

**Code:**

```
import sys

def tsp(graph, start):
    n = len(graph)
    visited = [False] * n
    visited[start] = True
    min_cost = sys.maxsize
    best_path = []

    def solve(curr, count, cost, path):
        nonlocal min_cost, best_path

        if count == n and graph[curr][start] > 0:
            total_cost = cost + graph[curr][start]

            if total_cost < min_cost:
                min_cost = total_cost
                best_path = path + [start]
            return

        for i in range(n):
            if not visited[i] and graph[curr][i] > 0:
                visited[i] = True
                solve(i, count + 1, cost + graph[curr][i], path + [i])
                visited[i] = False

    solve(start, 1, 0, [start])

    print("Minimum Cost:", min_cost)
    print("Path:", " -> ".join(map(str, best_path)))

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

tsp(graph, 0)
```

---

# Experiment 7

**Aim:** Write a Python program to implement Hill Climbing and Steepest Ascent Hill Climbing Algorithm.

**Code:**

```
def hill_climbing(graph, start, heuristic):
```

```
    current = start
    path = [current]

    while True:
        neighbors = graph[current]
        next_node = None

        for neighbor in neighbors:
            if heuristic[neighbor] < heuristic[current]:
                next_node = neighbor
                break

        if next_node is None:
            break

        current = next_node
        path.append(current)

    print("Hill Climbing Path:", " -> ".join(path))

def steepest_ascent_hill_climbing(graph, start, heuristic):
    current = start
    path = [current]

    while True:
        neighbors = graph[current]
        best_neighbor = current

        for neighbor in neighbors:
            if heuristic[neighbor] < heuristic[best_neighbor]:
                best_neighbor = neighbor

        if best_neighbor == current:
            break

        current = best_neighbor
        path.append(current)

    print("Steepest Ascent Path:", " -> ".join(path))
```

---

# Experiment 8

**Aim:** Write a Python program to load data in Google Colab notebook from CSV file.

**Code:**

```
import pandas as pd
from google.colab import files

uploaded = files.upload()

for file_name in uploaded.keys():
    df = pd.read_csv(file_name)

    print("File Loaded:", file_name)
    print(df.head())
```

---

# Experiment 9

**Aim:** Write a Python program to implement various Pandas commands on Titanic dataset.

**Code:**

```
import pandas as pd

df = pd.read_csv("Titanic-Dataset.csv")

print(df.head())
print(df.tail())
print(df.info())
print(df.describe())
print(df.columns)
print(df[['Name', 'Age', 'Fare']].head())
```

```
print(df[df['Age'] > 30].head())
print(df[df['Survived'] == 1].head())
print(df.groupby('Pclass').mean(numeric_only=True))
print(df.isnull().sum())

df['Age'] = df['Age'].fillna(df['Age'].mean())

print(df.sort_values(by='Fare', ascending=False).head())
print(df['Embarked'].unique())
print(df['Survived'].value_counts())
```

---

# Experiment 10

**Aim:** Write a Python program to integrate Python and SQL.

**Code:**

```
import sqlite3

conn = sqlite3.connect("student.db")
cur = conn.cursor()

cur.execute('''
CREATE TABLE IF NOT EXISTS students(
    id INTEGER PRIMARY KEY,
    name TEXT,
    age INT,
    marks REAL
)
''')

cur.executemany(
    "INSERT INTO students(name, age, marks) VALUES(?,?,?)",
    [('Aman',20,85), ('Riya',21,90), ('Karan',19,78)]
)

conn.commit()

print(cur.execute("SELECT * FROM students").fetchall())

cur.execute("UPDATE students SET marks=88 WHERE name='Aman'")
cur.execute("DELETE FROM students WHERE name='Karan'")

conn.commit()

print(cur.execute("SELECT * FROM students").fetchall())

conn.close()
```

---

# Experiment 11

**Aim:** Write a Python program to build a Basic Linear Regression Model.

**Code:**

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 6, 8, 10])

model = LinearRegression()
model.fit(X, y)

print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)

y_pred = model.predict(X)

print("Predictions:", y_pred)

new_value = np.array([[6]])
print("Prediction for 6:", model.predict(new_value)[0])
```

# Experiment 12

**Aim:** Write a Python program to implement Data Visualization techniques such as plotting bar graph, histogram, pie charts using Matplotlib.

**Code:**

```
import matplotlib.pyplot as plt

categories = ['A', 'B', 'C', 'D']
values = [10, 20, 15, 25]

plt.bar(categories, values)
plt.title("Bar Graph")
plt.xlabel("Categories")
plt.ylabel("Values")
plt.show()

data = [12, 15, 13, 17, 18, 19, 21, 22, 25, 27]

plt.hist(data, bins=5)
plt.title("Histogram")
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.show()

labels = ['Python', 'Java', 'C++', 'Others']
sizes = [40, 25, 20, 15]

plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title("Pie Chart")
plt.show()
```